

Workshop: Příprava dat a Feature Engineering

Transformace datasetu Titanic v Pythonu a Orange

Vojtěch Ondryhal

LS 2026

Metodický rámec a struktura workshopu

Cíle:

- Aplikovat pokročilé techniky čištění, transformace a inženýrství příznaků.
- Vybudovat pipeline v Pythonu s důrazem na zamezení úniku dat (*Data Leakage*).
- Konstruovat transformační preprocessory v nástroji Orange Data Mining.

Změna paradigmatu: Oproti EDA do dat aktivně zasahujeme s cílem maximalizovat predikční schopnost budoucích modelů.

Fáze analytického procesu

1. Programová transformace (Python)

Implementace úloh 1–5 s využitím `scikit-learn` a `pandas`. Striktní separace fází `.fit()` a `.transform()`.

2. Vizuální pipeline (Orange)

Sestavení toku dat pomocí widgetů pro preprocessing, diskretizaci a sampling.

3. Verifikace a Syntéza

Kontrola distribucí po transformaci, porovnání artefaktů a dokumentace v Jupyter Notebooku.

Důraz je kladen na pochopení dopadů transformací na strukturu dat, nikoliv na slepé spouštění funkcí.

Dataset Titanic: Výchozí stav pro Preprocessing

Stejně jako v předchozí úloze.

Identifikované inženýrské výzvy:

- embarked / sex: Textové řetězce (Nutný encoding).
- pclass: Číselný kód, ale reprezentuje pořadí (Nutný korektní přístup).
- fare: Extrémně zešikmená distribuce s dlouhým chvostem a outliers (Nutné robustní měřítko).
- age: Spojitý věk se šumem (Vhodný pro ukázkou binningu).
- survived: Silně nevyvážená cílová třída při malých výběrech.

Příprava základu

```
1 import pandas as pd
2 import seaborn as sns
3 from sklearn.model_selection import
  ↳ train_test_split
4
5 # Načtení dat a odstranění duplicitních
  ↳ sloupců z EDA
6 df = sns.load_dataset('titanic')
7 df = df.drop(columns=['alive', 'class',
  ↳ 'who', 'adult_male', 'deck',
  ↳ 'embark_town'])
8
9 # Export výchozího stavu pro Orange
10 df.to_csv('titanic_raw.csv', index=False)
```

Poznámka k Orange: Načtěte `titanic_raw.csv`. Pro úlohy transformace budete využívat widgety Preprocess, Continuize, Discretize a Data Sampler.

Úloha 1: Škálování příznaků a imunita vůči outlierům

Zadání:

1. Rozdělte data na trénovací a testovací část (80 : 20).
2. Aplikujte MinMaxScaler, StandardScaler a RobustScaler na sloupec fare.
3. **Úkol:** Dodržte pravidlo *No Data Leakage* — parametry škálovačů se smí učit pouze z trénovací množiny.
4. **Analytická otázka:** Vyneste detailní distribuci (zoom) přeshkálovaných hodnot pro běžné jízdenky (v rozsahu hodnot -1 až $+2$). Který škálovač jako jediný zachoval vnitřní rozlišení dat a proč?

Pomůcka

```
1  from sklearn.preprocessing import RobustScaler
2
3  X_train, X_test =
   ↪  train_test_split(df[['fare']],
   ↪  test_size=0.2, random_state=42)
4
5  scaler = RobustScaler()
6  # Správná separace učení a aplikace
7  X_train_scaled = scaler.fit_transform(X_train)
8  X_test_scaled = scaler.transform(X_test)
9
10 # Sledujte hodnoty .center_ a .scale_
```

Úloha 2: Strategie filtrace a ošetření chybějících hodnot

Zadání:

1. Zjistěte přesný podíl chybějících hodnot u všech proměnných v trénovací množině.
2. **Pravidlo filtrace:** Navrhněte a implementujte kód, který z datasetu automaticky vyřadí (dropne) sloupce, které mají více než **70 %** chybějících dat. Který sloupec toto kritérium splnil?
3. U sloupce embarked identifikujte řádky s chybějící hodnotou a odstraňte pouze tyto konkrétní záznamy.
4. **Analytická otázka:** Na základě chování chybějících hodnot u atributů age a deck rozhodněte, který z nich vykazuje **MAR** (Missing at Random) a který **MNAR** (Missing Not at Random). Jak se musí lišit vaše strategie pro jejich ošetření?

Pomůcka

```
1  # Výpočet limitu pro dropování sloupců
2  limit = len(X_train) * 0.3
3
4  # Dropování sloupců s vysokou mírou
   ↳ chybovosti
5  X_train_filtered = X_train.dropna(
6  thresh=limit, axis=1
7  )
8
9  # Odstranění řádků na základě konkrétního
   ↳ sloupce
10 X_train_clean = X_train.dropna(
11 subset=['embarked']
12 )
13
14 # V Orange využijte widget 'Purge Domain'
15 # nebo 'Impute' nastavený na 'Remove
   ↳ rows'
```

Úloha 3: Teorie vzorkování a reprezentativnost (SRS vs. Stratifikace)

Zadání:

1. Definujte globální proměnnou `size = 100` vyjadřující velikost požadovaného vzorku.
2. Nasimulujte náhodný výběr bez vracení (**SRSWOR**) a náhodný výběr s vracením (**SRSWR**). U verze s vracením exaktně kvantifikujte počet duplicitních řádků.
3. Aplikujte **Stratifikovaný výběr** vzhledem k cílové proměnné `survived`.
4. **Analytická otázka:** Porovnejte finální distribuci tříd (přežil/nepřežil) u všech tří metod. Proč prostá náhoda selhává a stratifikace přináší stabilitu?

Pomůcka

```
1 size = 100
2 # Metoda A: SRSWOR
3 s_wor = df.sample(n=size, replace=False)
4
5 # Metoda B: SRSWR
6 s_wr = df.sample(n=size, replace=True)
7 duplicity = size - s_wr.index.nunique()
8
9 # Metoda C: Stratifikace
10 _, s_strat = train_test_split(
11     df, test_size=size,
12     stratify=df['survived']
13 )
```

Úloha 4: Transformace kategorií (One-Hot vs. Ordinal Encoding)

Zadání:

1. Identifikujte rozdíl mezi charakterem proměnných sex (nominální) a pclass (ordinální).
2. Aplikujte OneHotEncoder na atribut embarked. Kolik nových sloupců vzniklo a proč?
3. Aplikujte OrdinalEncoder na proměnnou pclass se striktním zachováním logiky tříd ($1 > 2 > 3$).
4. **Analytická otázka:** Jaké riziko podstupujeme, pokud bychom nominální textový sloupec (embarked) zakódovali pouze jako prostá celá čísla 0, 1, 2?

Pomůcka

```
1  from sklearn.preprocessing import OneHotEncoder,  
   ↪ OrdinalEncoder  
2  
3  # One-Hot pro nominální data  
4  ohe = OneHotEncoder(sparse_output=False, dtype=int)  
5  emb_encoded =  
   ↪ ohe.fit_transform(df[['embarked']].dropna())  
6  
7  # Ordinal pro strukturovaná data  
8  # (U pclass definujeme inverzní pořadí)  
9  oe = OrdinalEncoder(categories=[[3, 2, 1]])  
10 df['pclass_encoded'] =  
   ↪ oe.fit_transform(df[['pclass']])
```

Úloha 5: Diskretizace (Binning), Vyhlazování a Tvorba příznaků

Zadání:

1. Rozdělte spojitý věk (age) na 3 přihrádky dvěma přístupy: podle **vzdálenosti** (*Equal-Width*) a podle **frekvence** (*Equal-Frequency*).
2. Proveďte vyhlazování dat (*Smoothing*): nahraďte surový věk průměrem jeho dané věkové přihrádky.
3. Vytvořte nový smysluplný příznak (**Feature Construction**) spojením informací o rodině: $\text{family_size} = \text{sibsp} + \text{parch} + 1$.
4. **Analytická otázka:** Pomocí scatter plotu vizualizujte dělicí čáry obou metod binningu. Jak reaguje Equal-Width a Equal-Frequency na nerovnoměrné shlukování dat?

Inženýrská nápověda

```
1  # Equal-Width
2  df['age_width'] = pd.cut(df['age'],
3  ↪  bins=3)
4
5  # Equal-Frequency
6  df['age_freq'] = pd.qcut(df['age'], q=3)
7
8  # Smoothing pomocí transformace průměru
9  df['age_smoothed'] =
10 ↪ df.groupby('age_width')['age']
11 ↪ .transform('mean')
12
13 # Feature Construction
14 df['family_size'] = df['sibsp'] +
15 ↪ df['parch'] + 1
```

Průvodce nástrojem Orange

Škálování, Chybějící hodnoty

Pro realizaci úloh v grafickém rozhraní Orange budete kombinovat uzly z palet **Data**, **Transform** a **Visualize**.

Úloha 1: Škálování a dělení dat

- Data Sampler: Rozdělí data na Train/Test (poměr 80 : 20).
- Preprocess → submodul **Normalize Features**: Zde přepínáte mezi metodami *MinMax*, *Standardize* a robustním *By span* (kvantily).

Úloha 2: Ošetření chybějících hodnot (Missing Values)

- Feature Statistics: Kontrola procentuálního zastoupení NaN hodnot ve sloupcích.
- Preprocess → submodul **Impute Missing Values**: Nastavení globální nebo lokální strategie doplňování dat.
- Purge Domain: Rychlé plošné odfiltrování řádků/sloupců splňujících kritérium chybovosti (nad 70 %).

Úloha 3: Teorie vzorkování (Sampling)

- Data Sampler: Podpora pro *Simple Random Sampling*, přepínač *Replacement* (s vracením) a fixní *Stratified* sampling podle cíle survived.

Úloha 4: Transformace kategorií (Encoding)

- Edit Domain: Klíčový uzel pro ordinální data (pclass). Umožňuje ručně přeuspořádat hierarchii textových hodnot před jejich číselným zakódováním.
- Continuize: Hlavní uzel pro One-Hot Encoding. Transformuje kategorické proměnné na multinomiální binární indikátory (0/1).

Úloha 5: Diskretizace a Feature Construction

- Discretize: Zajišťuje binning spojitých proměnných. Obsahuje exaktní přepínače *Equal-width intervals* (vzdálenost) a *Equal-frequency intervals* (frekvence / kvantily).
- Feature Constructor: Editor pro tvorbu nových sloupců. Umožňuje zapsat matematický výraz pro sloučení stávajících atributů (např. `sibsp + parch + 1` pro `family_size`).
- Scatter Plot: Závěrečné ověření hustoty, shluků a dělících čar po diskretizaci.

Metodika odevzdání a struktura výstupu

- **Formát odevzdání:** Soubor Jupyter Notebook (`.ipynb`) se zafixovanými výstupy buněk.
- **Termín:** Dle instrukcí v IS (standardně do konce cvičení).

Struktura dokumentu (Hierarchie nadpisů)

- `# Úloha X`: Každá z pěti úloh uvozena nadpisem 1. úrovně.
- `## Python Preprocessing`: Zdrojový kód pipeline, kontrolní výpisy polí, vizualizace transformovaných distribucí.
- `## Orange Workflow`: Snímky sestavené schémata widgetů (zejména detaily nastavení uzlu Preprocess a Data Sampler).

Požadavky na kritickou interpretaci

- **Inženýrské zdůvodnění:** Proč byla vybrána daná transformační matice nebo měřítko.
- **Syntéza artefaktů:** Porovnání, jak identickou operaci (např. One-Hot encoding nebo Stratifikaci) vizuálně reprezentuje Orange a jak čistě funkcionálně Python.
- **Závěrečný audit:** Krátké zhodnocení, v jakém stavu se dataset nacházel na začátku a jak je nyní optimálně připraven pro nasazení algoritmů strojového učení.